

LLM Agent

COMP4551

Binhang Yuan

AI Agent

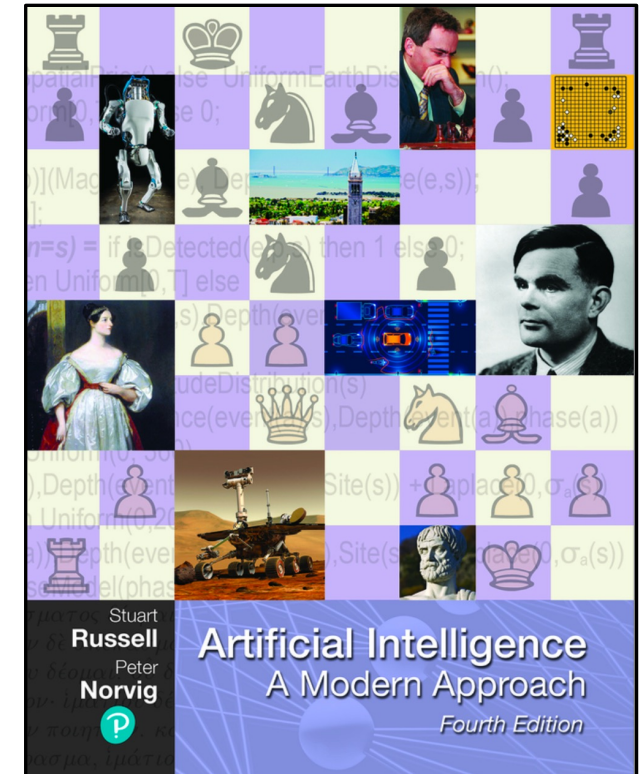
A Little Background about AI

- What is AI?
- *“The automation of activities that we associate with human thinking, activities such as decision-making, problem solving, learning ...”* – Richard Bellman
- *“AI is the science of making machines do things that would require intelligence if done by men.”* – Marvin Minsky



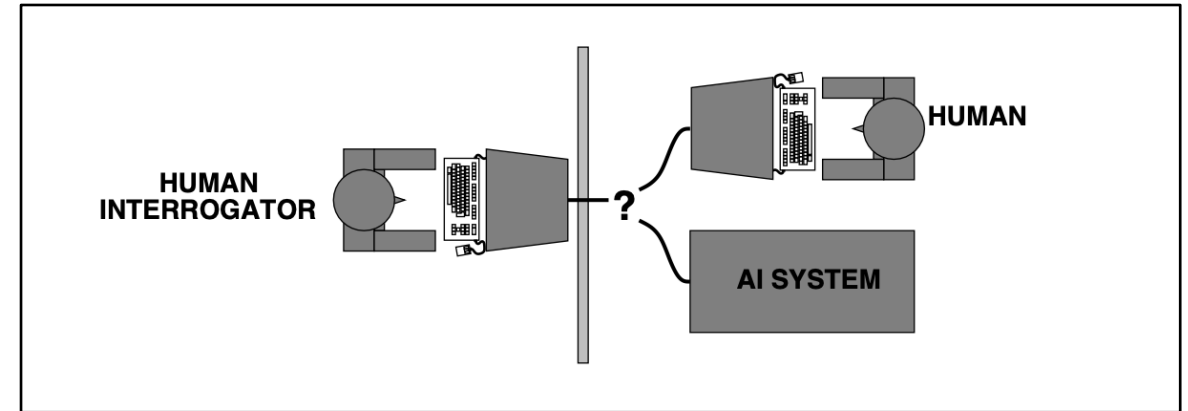
A Little Background about AI

- What is AI?
- Four possible combinations from two dimensions:
 - Human vs. Rational
 - Thought vs. Behaviour
- The view of AI falls into four categories:
 - Acting humanly
 - Thinking humanly
 - Thinking rationally
 - Acting rationally



What is AI? Acting Humanly

- *The Turing test:*
 - A thought experiment of “Can a machine think?”
 - “Can machines behave intelligently?”
 - A computer passes the test if a human interrogator, after posing some written questions, cannot tell whether the written responses come from a person or from a computer.
- Problem: Turing test is not reproducible, constructive, or amenable to mathematical analysis.
- Can ChatGPT pass the Turing test?
 - <https://humsci.stanford.edu/feature/study-finds-chatgpts-latest-bot-behaves-humans-only-better>



What is AI? Thinking Humanly

- *Cognitive science:*
- Require some scientific theories of internal activities of the brain:
 - What level of abstraction? Knowledge or circuits?
 - How to validate? Requires:
 - Predicting and testing the behaviour of human subjects (top-down);
 - or Direct identification from neurological data (bottom-up).
- Both approaches (roughly, Cognitive Science and Cognitive Neuroscience) are now distinct from AI.

What is AI? Thinking Rationally

- *Law of Thought:*
- Logic:
 - These laws of thought were supposed to govern the operation of the mind;
 - “Socrates is a man” and “all men are mortal” $\Rightarrow$ “Socrates is mortal”.
- Probability:
 - Allow rigorous reasoning with uncertain information.
- Problems:
 - Not all intelligent behaviour is mediated by logical deliberation;
 - What is the purpose of thinking? What thoughts should I have?

What is AI? Acting Rationally

- *A rational agent:*
 - Rational behaviour: do the right thing.
 - The right thing is expected to maximize goal achievement, given the available information.
 - Doesn't necessarily involve thinking, e.g., blinking reflex, but thinking should be in the service of rational action.
- Formally, an agent is an entity that perceives and acts, which can be defined as a function from percept histories to actions:
 - $f: P^* \rightarrow A$
- From any given class of environments and tasks, we seek the agent (or class of agents) with the best performance.

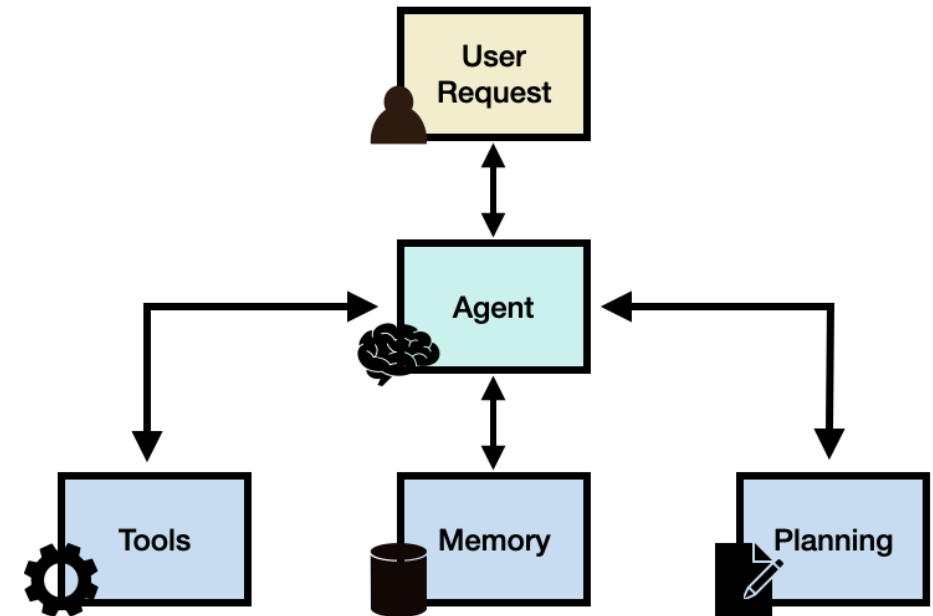
Single-Agent vs. Multi-Agent Frameworks

- **Single-Agent Framework:**
 - A single-agent system utilizes one AI entity to manage all tasks, encompassing planning, execution, and decision-making.
- **Multi-Agent Framework:**
 - A multi-agent system comprises multiple AI agents, each specialized in distinct tasks, collaborating to achieve a common goal.

LLM Agent

LLM Agent Framework

- LLM applications that can execute complex tasks:
 - Through the use of LLMs;
 - And other key modules like planning and memory.
- The architecture of an LLM Agent framework can be very flexible, we just provide one simple while clear categorization, which includes:
 - **User Request**: a user question or request;
 - **Agent/Brain**: the agent core acting as coordinator;
 - **Planning**: assists the agent in planning future actions;
 - **Memory**: manages the agent's past behaviours;
 - **Tool**: interacts with external environments.



LLM Agent Framework - Agent

- A large language model (LLM) with general-purpose capabilities serves as the main brain, agent module, or coordinator of the system.
- This component will be activated using a ***prompt template*** that entails important details about how the agent will operate, and the tools it will have access to (along with tool details).
- While not mandatory, an agent can be profiled or be assigned a persona to define its role.
- This profiling information is typically written in the prompt which can include specific details like role details, personality, social information, and other demographic information.

LLM Agent Framework - Planning

- Planning without feedback:
 - Leverage an LLM to decompose a detailed plan;
 - LLM break down the necessary steps or subtasks;
 - Then solve individually to answer the user request.
- Planning with feedback:
 - We can leverage a mechanism that enables the model to iteratively reflect and refine the execution plan based on past actions and observations.
 - The goal is to correct and improve on past mistakes which helps to improve the quality of final results.

Planning: ReAct

- Reasoning and Acting interleaved.
- **Core idea:** On every turn, the LLM emits a **thought** that justifies the next **action**. The environment returns an **observation**, which feeds back into the next thought. The loop ends when the model emits a **final Answer**.
- **Key properties:**
 - Adapts after every observation.
 - No global plan — purely local decisions.
 - One LLM call per thought/action step.

Published as a conference paper at ICLR 2023

REACT: SYNERGIZING REASONING AND ACTING IN LANGUAGE MODELS

Shunyu Yao^{*1}, Jeffrey Zhao², Dian Yu², Nan Du², Izhak Shafran², Karthik Narasimhan¹, Yuan Cao²

¹Department of Computer Science, Princeton University

²Google Research, Brain team

¹{shunyuy, karthikn}@princeton.edu

²{jeffreyzhao, dianyu, dunan, izhak, yuancao}@google.com

ABSTRACT

While large language models (LLMs) have demonstrated impressive performance across tasks in language understanding and interactive decision making, their abilities for reasoning (e.g. chain-of-thought prompting) and acting (e.g. action plan generation) have primarily been studied as separate topics. In this paper, we explore the use of LLMs to generate both reasoning traces and task-specific actions in an interleaved manner, allowing for greater synergy between the two: reasoning traces help the model induce, track, and update action plans as well as handle exceptions, while actions allow it to interface with and gather additional information from external sources such as knowledge bases or environments. We apply our approach, named *ReAct*, to a diverse set of language and decision making tasks and demonstrate its effectiveness over state-of-the-art baselines in addition to improved human interpretability and trustworthiness. Concretely, on question answering (HotpotQA) and fact verification (Fever), *ReAct* overcomes prevalent issues of hallucination and error propagation in chain-of-thought reasoning by interacting with a simple Wikipedia API and generating human-like task-solving trajectories that are more interpretable than baselines without reasoning traces. Furthermore, on two interactive decision making benchmarks (ALFWorld and WebShop), *ReAct* outperforms imitation and reinforcement learning methods by an absolute success rate of 34% and 10% respectively, while being prompted with only one or two in-context examples.

1 INTRODUCTION

A unique feature of human intelligence is the ability to seamlessly combine task-oriented actions with verbal reasoning (or inner speech, Alderson-Day & Fernyhough, 2015), which has been theorized to play an important role in human cognition for enabling self-regulation or strategization (Vygotsky, 1987; Luria, 1965; Fernyhough, 2010) and maintaining a working memory (Baddeley, 1992). Consider the example of cooking up a dish in the kitchen. Between any two specific actions, we may reason in language in order to track progress (“now that everything is cut, I should heat up the pot of water”), to handle exceptions or adjust the plan according to the situation (“I don’t have salt, so let me use soy sauce and pepper instead”), and to realize when external information is needed (“how do I prepare dough? Let me search on the Internet”). We may also act (open a cookbook to read the recipe, open the fridge, check ingredients) to support the reasoning and to answer questions (“What dish can I make right now?”). This tight synergy between “acting” and “reasoning” allows humans to learn new tasks quickly and perform robust decision making or reasoning, even under previously unseen circumstances or facing information uncertainties.

Recent results have hinted at the possibility of combining verbal reasoning with interactive decision making in autonomous systems. On one hand, properly prompted large language models (LLMs) have demonstrated emergent capabilities to carry out several steps of reasoning traces to derive

^{*}Work during Google internship. Project page with code: <https://react-lm.github.io/>.

Planning: ReAct Example

Task: What is the population of the capital of France?

Thought 1

I need the capital of France first.

Action 1

`Search["capital of France"]`

Observation 1

Paris is the capital and most populous city of France.

Thought 2

Now I need the population of Paris.

Action 2

`Search["population of Paris"]`

Observation 2

Paris has a population of about 2.1 million (city proper).

Final Answer

About 2.1 million people.

Each step is decided after seeing the previous observation — no pre-made plan.

Planning: Plan-and-Execute

- **Core idea:** A **planner** (usually a strong LLM) first turns the task into an ordered list of sub-steps. An **executor** then carries out each step in turn — calling tools as needed. If a step fails or yields a surprise, the planner can re-plan the remaining work.
- **Key properties:**
 - An explicit global plan, which is easier to inspect.
 - The executor can be a cheaper/smaller model.
 - Independent steps can run in parallel.

Plan-and-Solve Prompting: Improving Zero-Shot Chain-of-Thought Reasoning by Large Language Models

Lei Wang¹ Wanyu Xu² Yihuai Lan³ Zhiqiang Hu³ Yunshi Lan⁴
Roy Ka-Wei Lee³ Ee-Peng Lim^{1*}

¹Singapore Management University

²Southwest Jiaotong University

³Singapore University of Technology and Design

⁴East China Normal University

Abstract

Large language models (LLMs) have recently been shown to deliver impressive performance in various NLP tasks. To tackle multi-step reasoning tasks, few-shot chain-of-thought (CoT) prompting includes a few manually crafted step-by-step reasoning demonstrations which enable LLMs to explicitly generate reasoning steps and improve their reasoning task accuracy. To eliminate the manual effort, Zero-shot-CoT concatenates the target problem statement with “Let’s think step by step” as an input prompt to LLMs. Despite the success of Zero-shot-CoT, it still suffers from three pitfalls: calculation errors, missing-step errors, and semantic misunderstanding errors. To address the missing-step errors, we propose Plan-and-Solve (PS) Prompting. It consists of two components: first, devising a plan to divide the entire task into smaller subtasks, and then carrying out the subtasks according to the plan. To address the calculation errors and improve the quality of generated reasoning steps, we extend PS prompting with more detailed instructions and derive PS+ prompting. We evaluate our proposed prompting strategy on ten datasets across three reasoning problems. The experimental results over GPT-3 show that our proposed zero-shot prompting consistently outperforms Zero-shot-CoT across all datasets by a large margin, is comparable to or exceeds Zero-shot-Program-of-Thought Prompting, and has comparable performance with 8-shot CoT prompting on the math reasoning problem. The code can be found at <https://github.com/AGI-Edgerunners/Plan-and-Solve-Prompting>.

1 Introduction

Large language models (LLMs) (Brown et al., 2020; Thoppilan et al., 2022; Chowdhery et al., 2022) have recently proven highly effective in various NLP tasks. Unlike the previous pre-trained language models (PTMs) (Devlin et al., 2019; Liu

*Corresponding author.

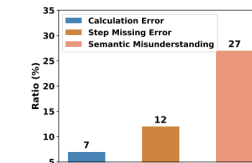


Figure 1: Error analysis of 46 GSM8K problems with incorrect answers returned by Zero-shot-CoT using GPT-3 LLM. Following Wei et al. (2022b) and Wang et al. (2022a), we assign “Calculation Error” (7%), “Step Missing Error” (12%), or “Semantic misunderstanding Error” (27%) to each incorrect answer.

et al., 2019), these LLMs are typically provided as a service, with no access to model parameters due to commercial considerations and potential risks of misuse (Sun et al., 2022). Thus, it is challenging to fine-tune LLMs for downstream tasks (He et al., 2021; Houlisby et al., 2019; Devlin et al., 2019). Instead, we leverage LLMs to solve complex reasoning problems by eliciting their strong reasoning abilities over their embedded knowledge using instructions (or trigger sentences). So far, LLMs have shown impressive abilities to solve new reasoning problems by simply conditioning them on a few illustrative examples (i.e., few-shot learning) or a prompt to solve new problems without illustrative examples (i.e., zero-shot learning).

To tackle multi-step complex reasoning tasks using LLMs, Wei et al. (2022b) proposes few-shot chain-of-thought (CoT) prompting, which enables LLMs to explicitly generate the intermediate reasoning steps before predicting the final answer with a few manual step-by-step reasoning demonstration examples. In (Kojima et al., 2022), Zero-shot CoT eliminates the need for manually crafted examples in prompts by appending “Let’s think step by step” to the target problem fed to LLMs such

Planning: Plan and Execute Example

Task: What is the population of the capital of France?

Phase 1 — Plan

Planner output:

- ① Find the capital of France.
- ② Look up the population of that city.
- ③ Return the population as the answer.



Phase 2 — Execute

- | | |
|----------|--------------------|
| ① Step 1 | → Paris |
| ② Step 2 | → ~2.1 million |
| ③ Final | 2.1 million people |

If an execution step fails, the Planner is invoked again to revise remaining steps.

Reasoning happens once, up front — the executor does not need to re-think the whole task.

LLM Agent Framework - Memory

- The memory module helps to store the agent's internal logs, including: past thoughts; past actions; observations from the environment; interactions between the agent and the user. There are also different memory formats to consider when building agents. Representative memory formats include: natural language, embeddings, structural data, e.g., relational database.
- **Short-term memory:** includes context information about the agent's current situations; this is typically realized by in-context learning, which means it is short and finite due to context window constraints.
- **Long-term memory:** includes the agent's past behaviors and thoughts that need to be retained and recalled over an extended period of time; this often leverages an external vector store accessible through fast and scalable retrieval to provide relevant information for the agent as needed.

LLM Agent Framework - Tool

- Tools correspond to a set of tool/s that enables the LLM agent to interact with external environments such as:
 - Wikipedia Search API;
 - Code Interpreter;
 - Math Engine;
 - Tools could also include databases, knowledge bases, and external models.
- When the agent interacts with external tools it executes tasks via workflows that assist the agent to obtain observations or necessary information to complete subtasks and satisfy the user request.

Agent Implementation by Agent Skills

Skills for Agent / Brain

Progressive disclosure lets the coordinator choose the right capability without bloating context

The coordination problem

The Brain must decide what to do for **every** incoming request.

Stuffing every possible procedure into the system prompt wastes context and slows the model down.

What it actually needs: a *menu* of capabilities it can browse, plus the ability to pull full instructions only when a capability is chosen.

1

Discovery

At startup the Brain loads only the name + description of each skill — just enough to recognise when one applies.

2

Activation

When a request matches a description, the Brain pulls the full SKILL.md body into context — instructions, examples, edge cases.

3

Execution

The Brain follows those instructions, loading referenced files or running bundled scripts as the task unfolds.

Takeaway: skills turn the Brain into a *router* over capabilities — cheap when idle, rich when needed.

Skills for Planning

Each SKILL.md is a vetted playbook — planning becomes following, not inventing

Without skills

Brain improvises a plan from scratch each time
Edge cases are hit, not anticipated
Format and ordering drift between runs
Same mistake recurs across users and sessions

With a matching skill

Step-by-step instructions already written by a human expert
Input/output examples anchor the plan
Common edge cases are pre-enumerated
Long procedures split across referenced files — planned, not improvised

SKILL.md — a planning playbook

name: pdf-processing

description: Extract PDF text, fill forms,
merge files. Use when handling PDFs.

Step-by-step instructions

1. Identify whether text is embedded or scanned
2. Use pdfplumber for extraction
3. If scanned, route through OCR first

Examples of inputs and outputs

Common edge cases

- encrypted PDFs - rotated pages
- multi-column layouts

The Brain doesn't plan these steps — the skill author already did.

Skills for Memory

Portable, version-controlled organisational memory — loaded only on demand

Memory the Brain actually needs

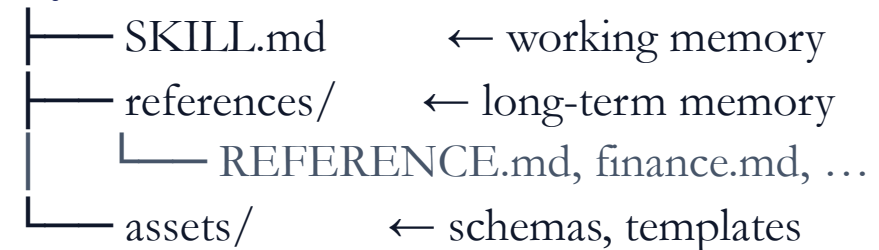
Procedural memory — how our team does X

Domain memory — terminology, schemas, regulations

User context — preferences, prior decisions, style guides

A skill folder = a memory shelf

my-skill/



Portable

A folder on disk. Ship it to any agent that supports the spec — no retraining, no migration.

Version-controlled

Lives in git alongside code. Diffs, reviews, rollbacks — the same hygiene as any other artefact.

On-demand

The references/ files load only when the skill is active, so this memory costs zero context until it is used.

Skills for Tool

scripts/ + allowed-tools bundle executable capabilities with the instructions that use them

scripts/

Executable code the agent can run

Python, Bash, or JavaScript files bundled inside the skill — self-contained, with documented dependencies and graceful error handling.

allowed-tools

Pre-approved tool list

A space-delimited field in the frontmatter. The skill author declares which tools are safe to invoke, so the agent doesn't have to ask per call.

assets/

Static resources the tools consume

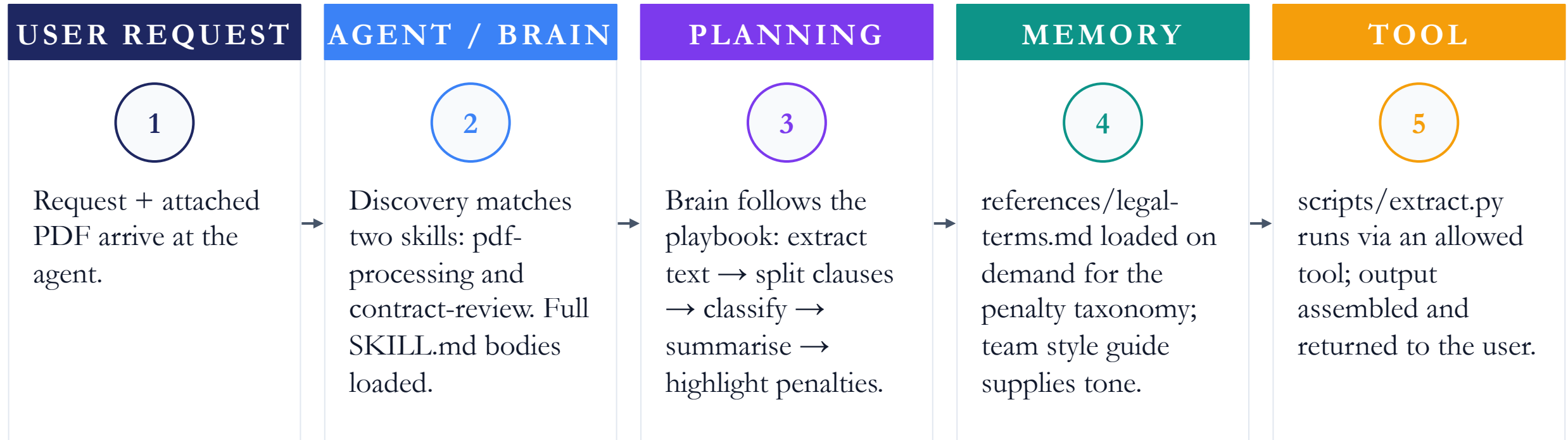
Document templates, lookup tables, schemas — inputs the scripts read and the Brain references by filename.

Why this matters: an ordinary *tool* is a raw capability (e.g. `run_python`). A skill packages a tool **with the know-how to use it well** — scripts + instructions + assets shipped together.

Agent Skills End-to-End Example

Tracing one user request through all five components, with skills wired in

Scenario *“Summarise this 40-page PDF contract and flag every clause that references termination penalties.”*

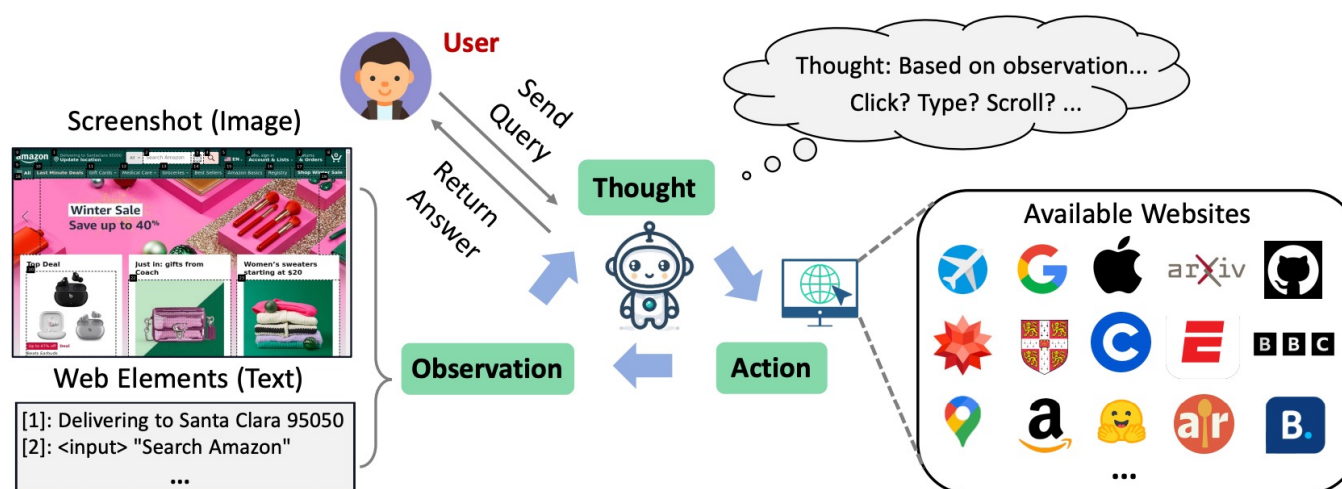


LLM Agent Case Study — WebVoyager

WebVoyager



RELAXED
SYSTEM LAB



WebVoyager takes web tasks assigned by a human and automatically browses the web online. At each step, WebVoyager selects actions based on screenshots and text (the ‘type’ of the web element and its contents). Once the task is completed, the answers will be returned to the user. For example, for a user query: "Find the cost of a 2-year protection for PS4 on Amazon.", the agent interacts with Amazon online, locates the PS4, identifies the 2-year protection price, and returns "\$30.99" to the user.

WebVoyager: Building an End-to-End Web Agent with Large Multimodal Models

Hongliang He^{1,3*}, Wenlin Yao², Kaixin Ma², Wenhao Yu², Yong Dai²,
Hongming Zhang², Zhenzhong Lan³, Dong Yu²

¹Zhejiang University, ²Tencent AI Lab, ³Westlake University
hehongliang@westlake.edu.cn, wenlinyao@global.tencent.com

Abstract

The rapid advancement of large language models (LLMs) has led to a new era marked by the development of autonomous applications in real-world scenarios, which drives innovation in creating advanced web agents. Existing web agents typically only handle one input modality and are evaluated only in simplified web simulators or static web snapshots, greatly limiting their applicability in real-world scenarios. To bridge this gap, we introduce WebVoyager, an innovative Large Multimodal Model (LMM) powered web agent that can complete user instructions end-to-end by interacting with real-world websites. Moreover, we establish a new benchmark by compiling real-world tasks from 15 popular websites and introduce an automatic evaluation protocol leveraging multimodal understanding abilities of GPT-4V to evaluate open-ended web agents. We show that WebVoyager achieves a 59.1% task success rate on our benchmark, significantly surpassing the performance of both GPT-4 (All Tools) and the WebVoyager (text-only) setups, underscoring the exceptional capability of WebVoyager. The proposed automatic evaluation metric achieves 85.3% agreement with human judgment, indicating its effectiveness in providing reliable and accurate assessments of web agents.¹

1 Introduction

The recent advancement of large language models (LLMs), such as ChatGPT and GPT-4 (OpenAI, 2023), have sparked significant interest in developing LLM-based autonomous agents (AutoGPT, 2022) for complex task execution (Qin et al., 2023; Schick et al., 2023). Recent studies have explored the construction of text-based web browsing environments and how to instruct large language model agents to perform web navigation (Nakano et al., 2021; Gur et al., 2023; Zhou et al., 2023; Lu et al.,

2023). The primary challenge in these works lies in managing complex and verbose HTML texts, and solutions include simplifying and structuring HTML (Nakano et al., 2021; Zhou et al., 2023; Gur et al., 2023; Deng et al., 2023).

However, existing approaches overlook a critical functionality of browsing: rendering HTML into visual webpages. Particularly, vision capability is crucial for utilizing tools such as web browsers, as rendered web pages are inherently designed with user experience (UX), emphasizing intuitive information and structured presentation. This design principle of rendering makes visual analysis more effective than mere HTML representation. At present, large multimodal models (LMMs), particularly GPT-4V (OpenAI, 2023) and Gemini (Team et al., 2023), demonstrate a remarkable ability to integrate intricate visual cues with textual information. Existing studies such as Pix2Struct (Lee et al., 2023) and WebArena (Zhou et al., 2023), have initiated explorations into using screenshots as inputs for decision-making in web navigation, yet these are preliminary and do not represent a deep exploration. Therefore, building multimodal web agents to leverage the environment rendered by browsers through screenshots, thus mimicking human web browsing behavior, is now a viable approach to enhance web navigation abilities.

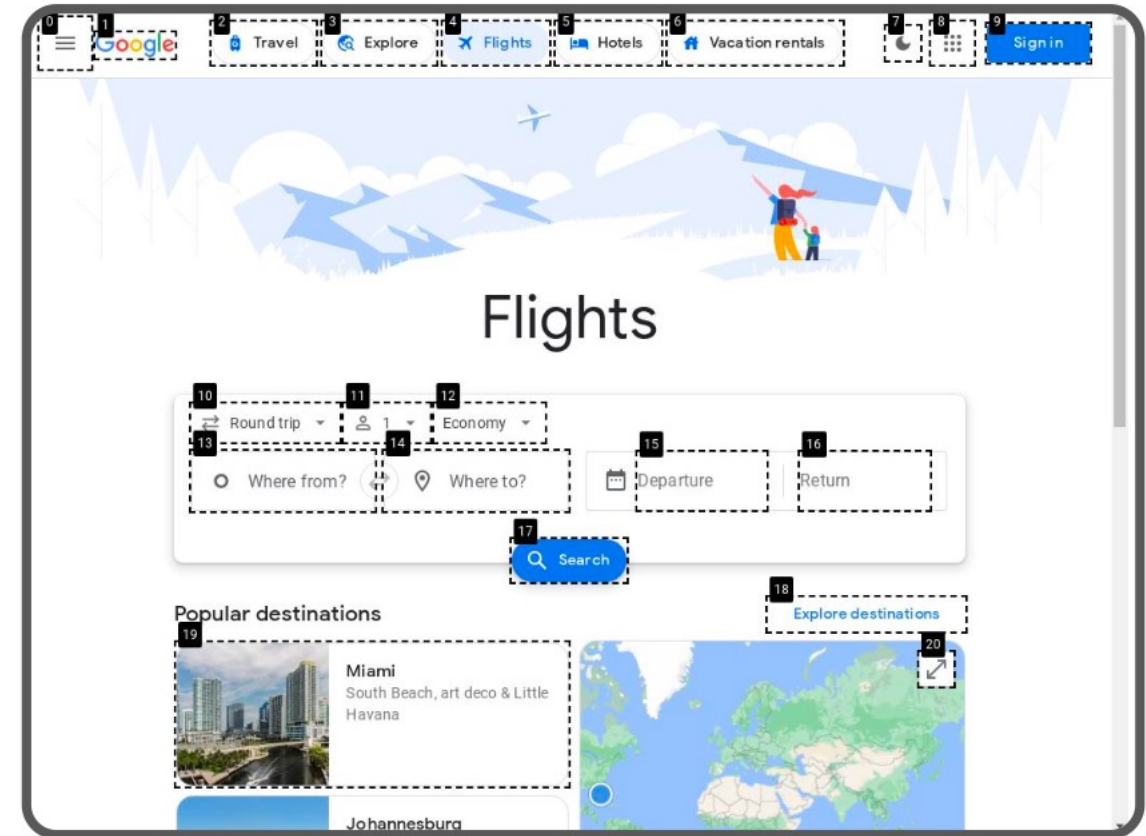
We introduce WebVoyager (Figure 1), a multimodal web agent designed to autonomously accomplish web tasks online from start to finish, managing the entire process end-to-end without any intermediate human intervention. WebVoyager processes the user query by making observations from screenshots and textual content in interactive web elements, formulates a thought on what action to take (such as clicking, typing, or scrolling, etc.), and then executes that action on the websites. Inspired by Set-of-Mark Prompting (Yang et al., 2023a), we mark interactive web elements on screenshots (see Figure 2) to facilitate decision-

^{*}Work done during the internship at Tencent AI Lab.
¹Our code and data will be released at <https://github.com/MinorJerry/WebVoyager>

6864

Observation Space

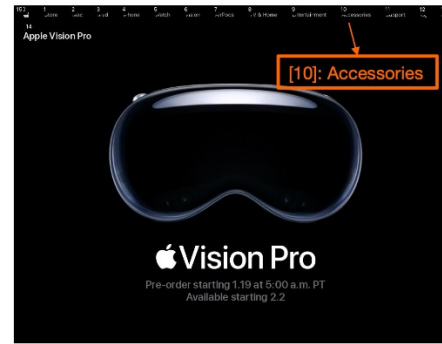
- Takes the visual information from the web(screenshots) as the primary source of input;
- Add borders to most of the interactive elements on the web pages and label them with numerical tags in the top left corner.



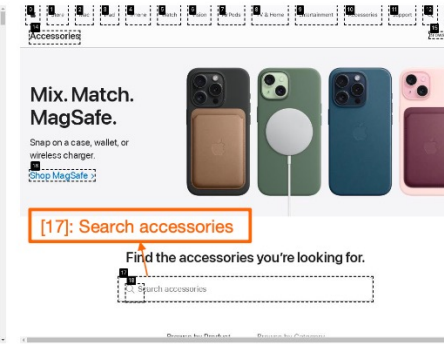
WebVoyager

Action Space:

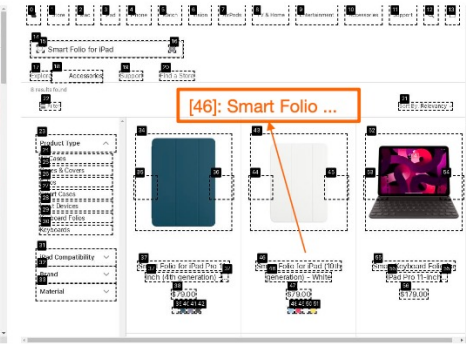
- Click;
- Input;
- Scroll;
- Wait;
- Back;
- Jump to Search Engine;
- Answer.



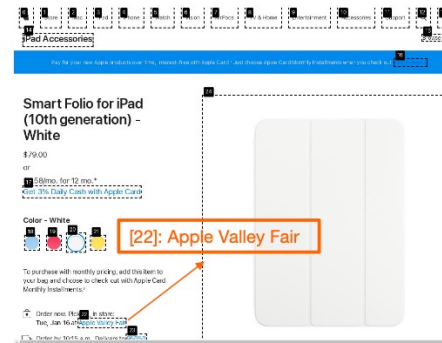
Step 1: Click [10]



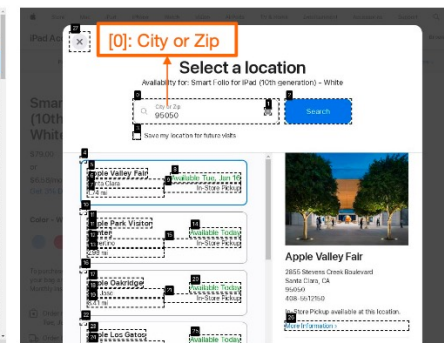
Step 2: Type [17]: Smart Folio for iPad



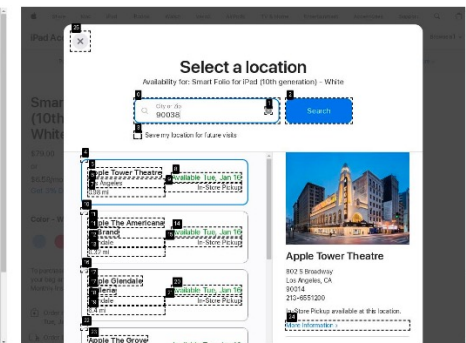
Step 3: Click [46]



Step 4: Click [22]



Step 5: Type [0]: 90038



Step 6: ANSWER

Given an instruction, WebVoyager:

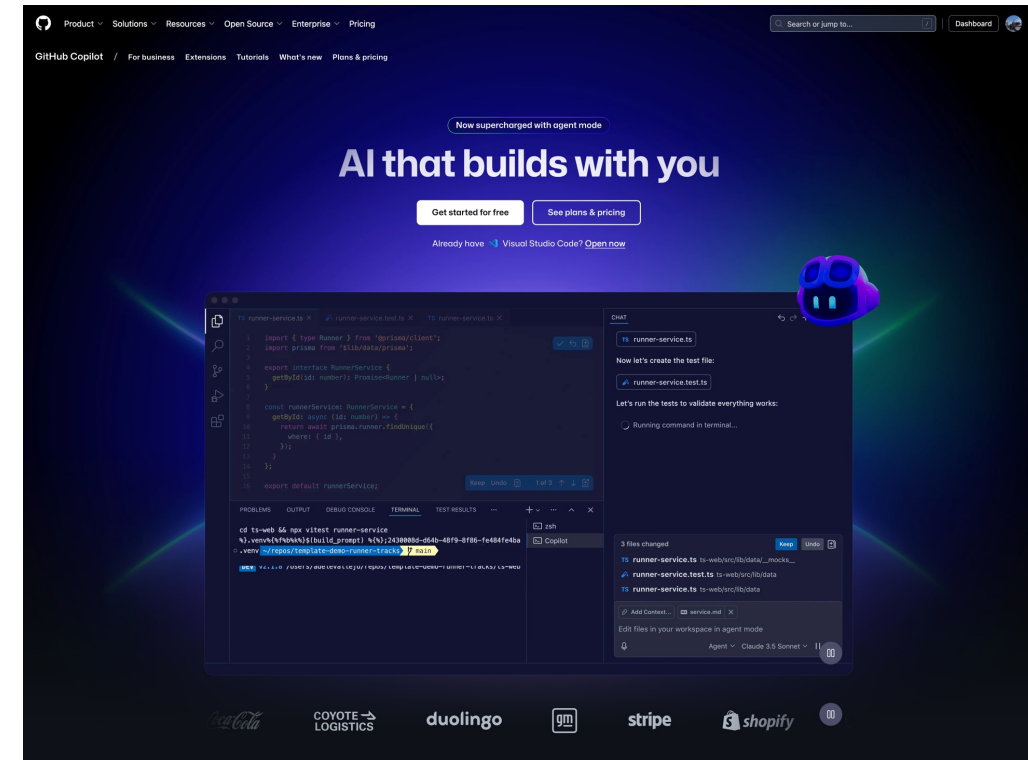
1. Instantiate a web browser.
2. Perform actions with visual (i.e., screenshots) and textual (i.e., HTML elements) signals from the web.
3. The agent produces an action based on the inputs at every step.
4. The action is then executed in the browser environment.

The process continues until the agent decides to stop.

LLM Agent Case Study — GitHub Copilot

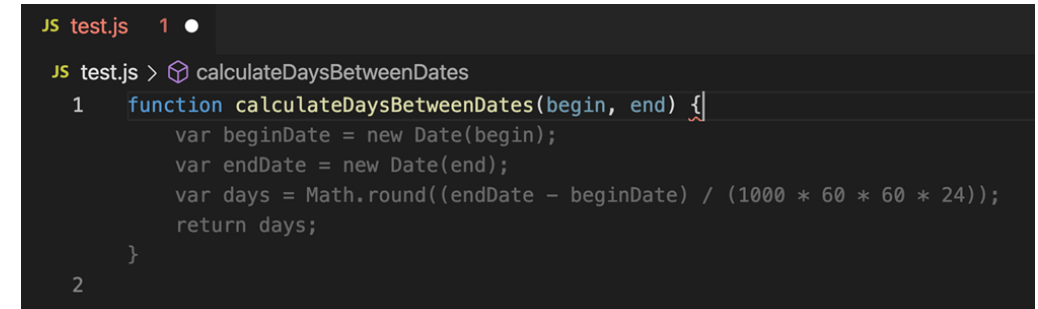
GitHub Copilot: Coding Assistant

- Copilot is an AI pair programmer tool in Visual Studio Code.
- Provide code suggestions as you type in the editor:
 - **AI Code completions:** start typing in the editor, and Copilot provides code suggestions based on your existing code that matches your coding style.
- Use natural language chat to ask about the code;
 - **Natural language chat:** use a conversational interface to ask about your codebase or make edits across your project. Switch between ask, edit, agent mode based on your needs.
- Start an editing session for implementing new feature and fixing bugs.
 - **Smart actions:** boost your developer productivity, from the terminal, source control operations, to debugging and testing code.



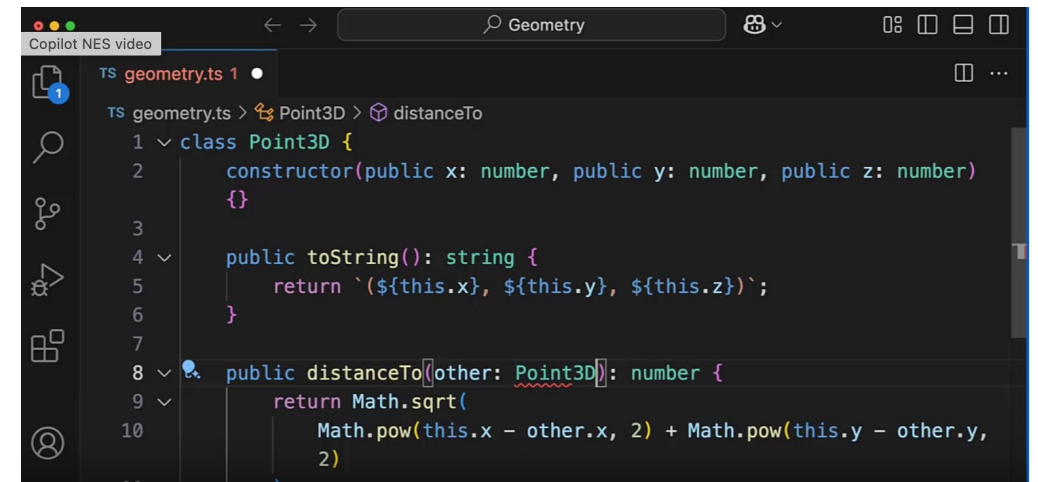
GitHub Copilot: AI Code Completions

- Two kinds of code completions:
- **Code completions**: Start typing in the editor, and Copilot provides code suggestions that match your coding style and take your existing code into account.
- **Next Edit Suggestions**: Predict your next code edit with Copilot Next Edit Suggestions, aka Copilot NES. Based on the edits you're making, Copilot NES both predicts the location of the next edit you'll want to make and what that edit should be.



```
JS test.js 1 •
JS test.js > calculateDaysBetweenDates
1 function calculateDaysBetweenDates(begin, end) {
    var beginDate = new Date(begin);
    var endDate = new Date(end);
    var days = Math.round((endDate - beginDate) / (1000 * 60 * 60 * 24));
    return days;
2 }
```

[Code Completions](#)

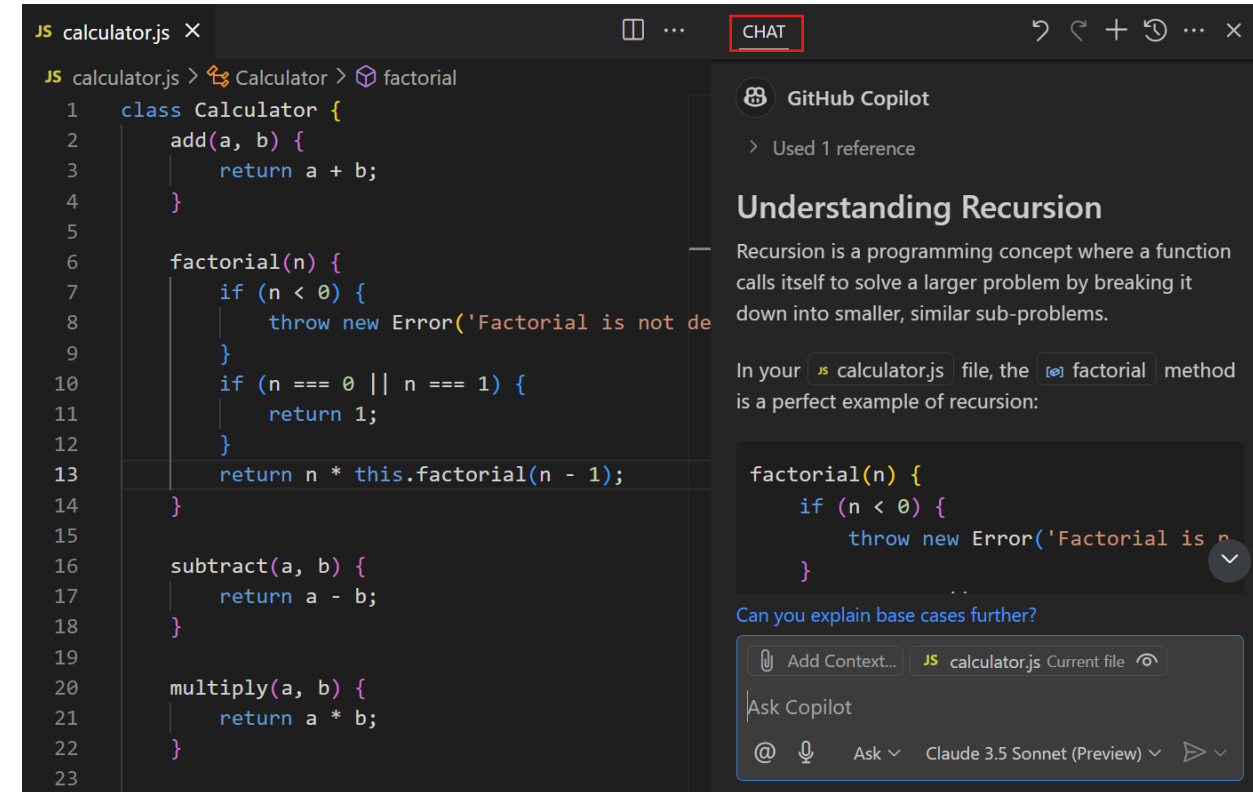


```
Copilot NES video
TS geometry.ts 1 •
TS geometry.ts > Point3D > distanceTo
1 class Point3D {
2     constructor(public x: number, public y: number, public z: number)
3     {}
4     public toString(): string {
5         return `${this.x}, ${this.y}, ${this.z}`;
6     }
7
8     public distanceTo(other: Point3D): number {
9         return Math.sqrt(
10             Math.pow(this.x - other.x, 2) + Math.pow(this.y - other.y,
11                 2)
12         );
13     }
14 }
```

[Next Edit Suggestions](#)

GitHub Copilot: Natural Language Chat

- Use chat in VS Code when you need to:
 - **Understand code:** "Explain how this authentication middleware works"
 - **Debug issues:** "Why am I getting a null reference in this loop?"
 - **Get code suggestions:** "Show me how to implement a binary search tree in Python"
 - **Optimize performance:** "Help me improve the efficiency of this database query"
 - **Learn best practices:** "What's the recommended way to handle errors in async functions?"
 - **Get VS Code tips:** "How do I customize keyboard shortcuts?"

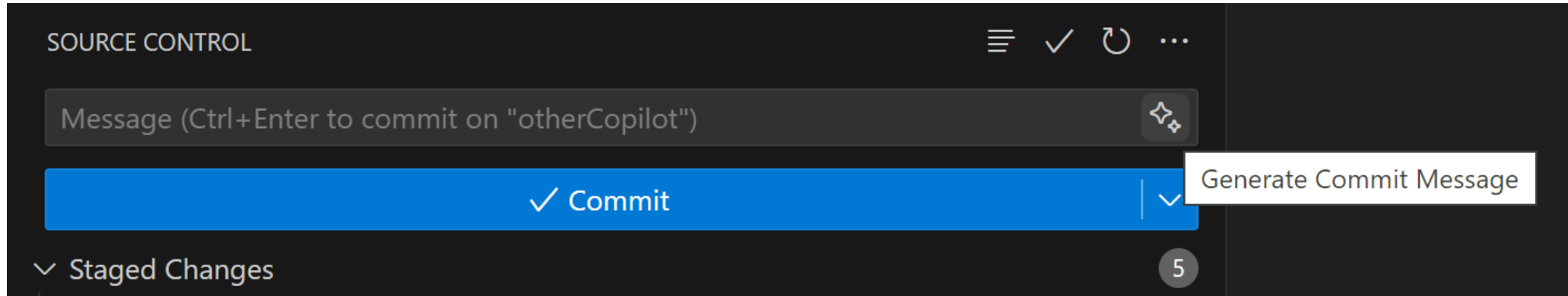


Natural Language Chat

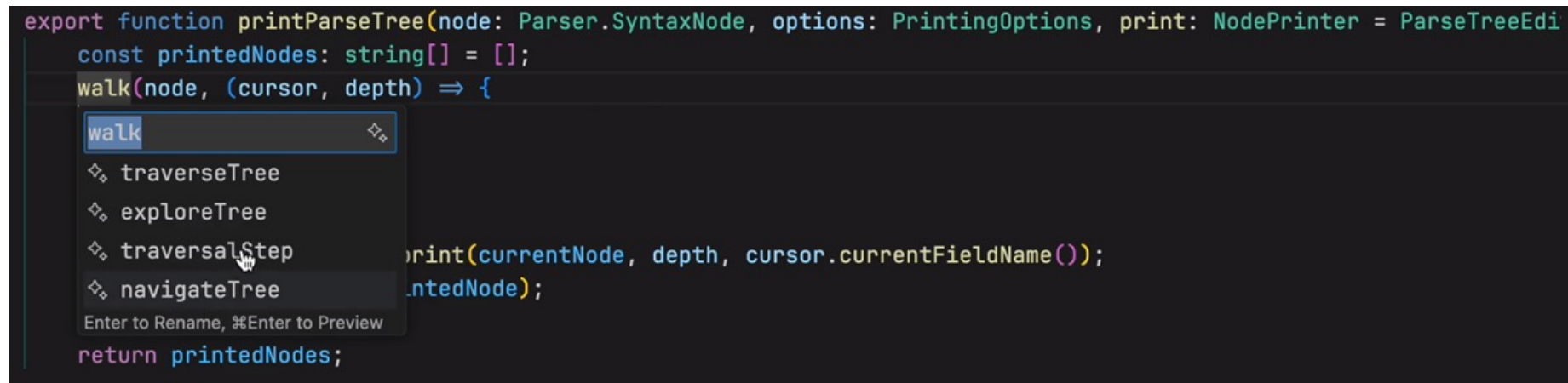
GitHub Copilot: Smart Actions

- For several common scenarios, you can use smart actions to get help from Copilot without having to write a prompt:
 - Generate a commit message and PR information;
 - Rename symbols;
 - Generate documentation;
 - Generate tests;
 - Explain code;
 - Fix coding errors;
 - Fix testing errors;
 - Fix terminal errors;
 - Review code;
 - Semantic search results.

GitHub Copilot: Smart Actions

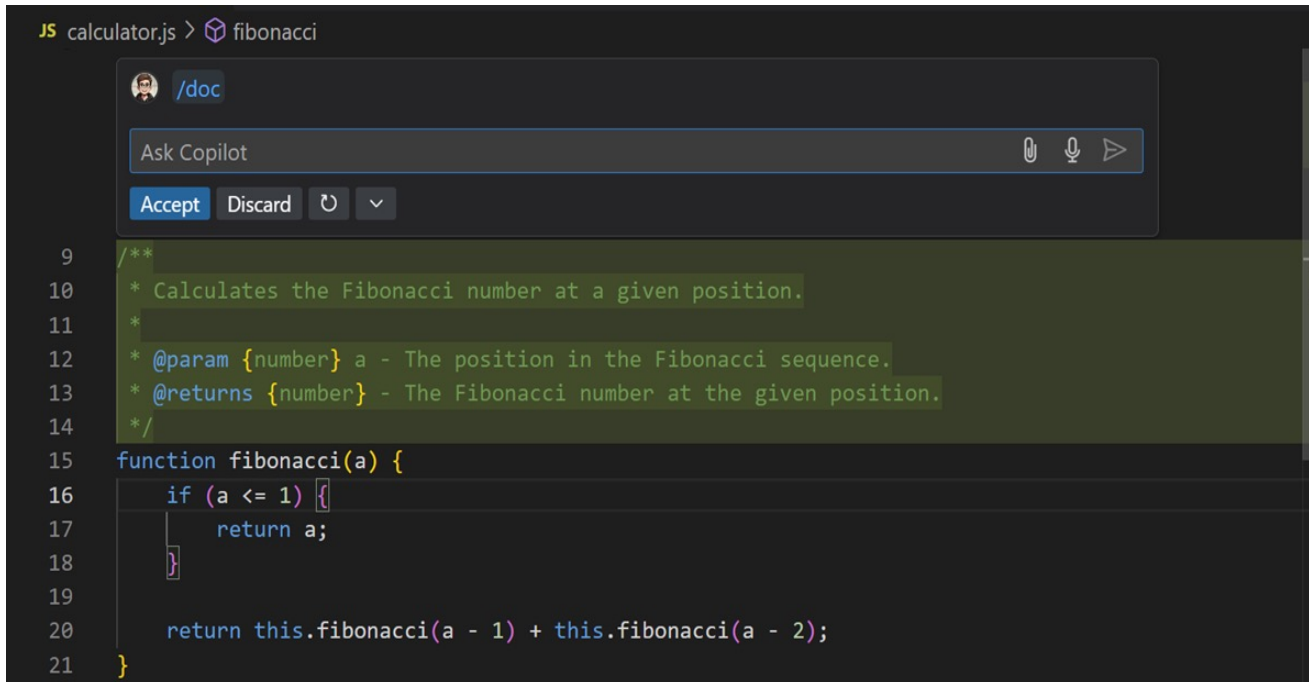


[Generate a commit message and PR information](#)



[Rename symbols](#)

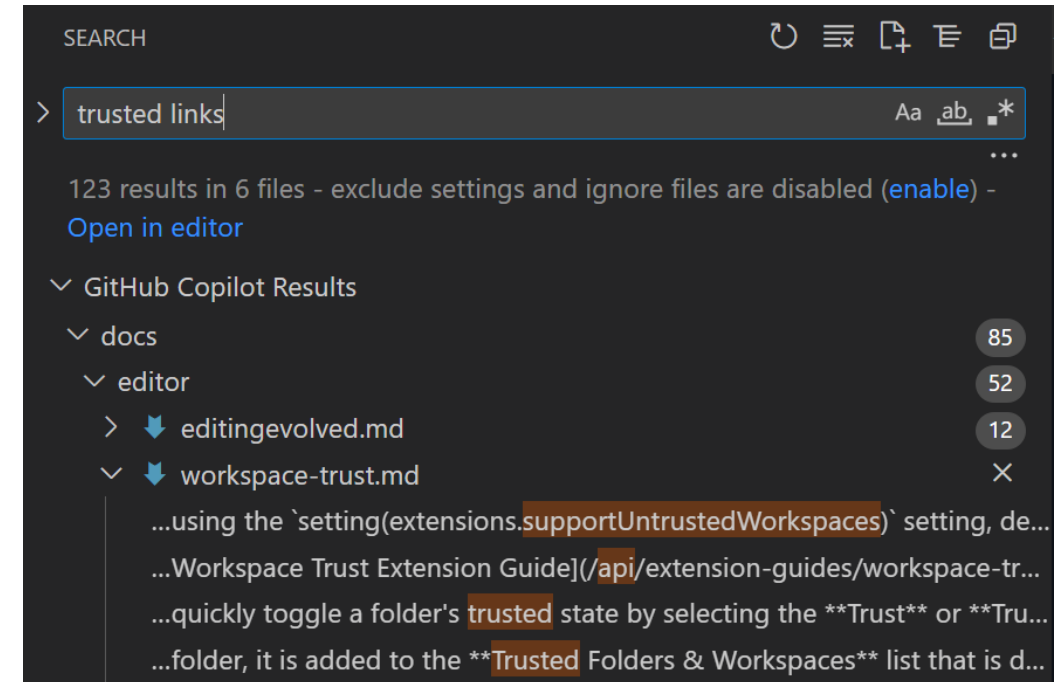
GitHub Copilot: Smart Actions



The screenshot shows a VS Code editor with a file named `calculator.js` containing a `fibonacci` function. A Copilot chat window is open with the prompt `/doc`. The chat interface includes an "Ask Copilot" input field and buttons for "Accept", "Discard", and a refresh icon. The generated documentation is displayed in a green box above the function code.

```
9 /**
10  * Calculates the Fibonacci number at a given position.
11  *
12  * @param {number} a - The position in the Fibonacci sequence.
13  * @returns {number} - The Fibonacci number at the given position.
14  */
15 function fibonacci(a) {
16   if (a <= 1) {
17     return a;
18   }
19
20   return this.fibonacci(a - 1) + this.fibonacci(a - 2);
21 }
```

[Generate documentation](#)



The screenshot shows the VS Code search interface with the query `trusted links`. It displays 123 results across 6 files. The results are categorized under "GitHub Copilot Results" and include files like `docs` and `editor`. The `workspace-trust.md` file is highlighted, showing search results related to the `supportUntrustedWorkspaces` setting.

SEARCH

> trusted links

123 results in 6 files - exclude settings and ignore files are disabled (enable) - [Open in editor](#)

GitHub Copilot Results

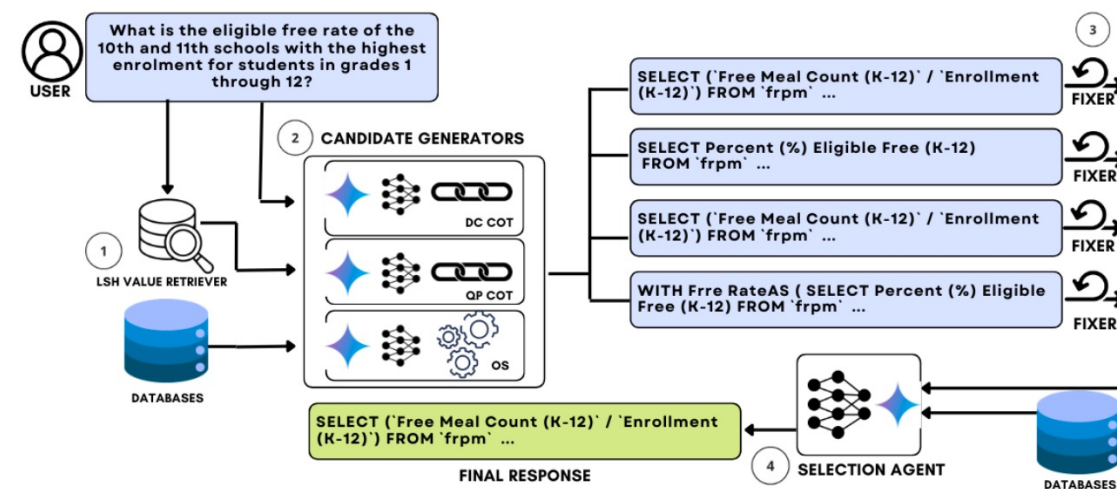
- docs (85)
- editor (52)
 - editingevolved.md (12)
 - workspace-trust.md (X)
 - ...using the `setting(extensions.supportUntrustedWorkspaces)` setting, de...
 - ...Workspace Trust Extension Guide[[/api/extension-guides/workspace-tr...](#)
 - ...quickly toggle a folder's **trusted** state by selecting the ****Trust**** or ****Tru...**
 - ...folder, it is added to the ****Trusted** Folders & Workspaces** list that is d...

[Semantic search results](#)

LLM Agent Case Study — CHASE-SQL

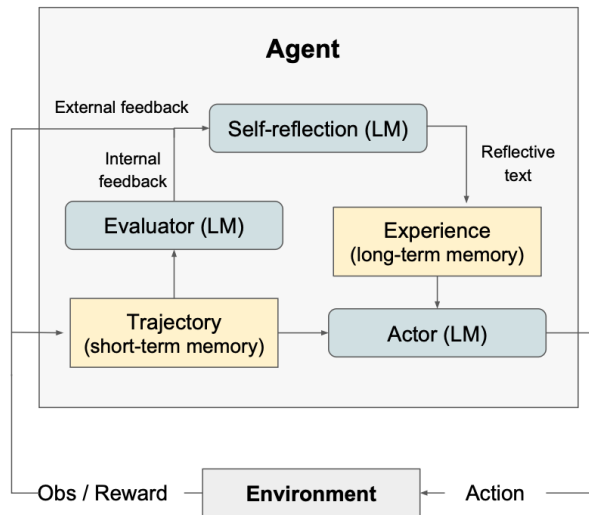
CHASE-SQL (a Multi-Agent Framework) breaks the task into modular sub-tasks, generate SQL, fix errors, and select the best result.

- 1. Value retrieval: Extract keywords from the given question using an LLM prompted. For each keyword, employ some measurement, i.e., locality-sensitive hashing, to retrieve the most syntactically-similar words in database schema.
- 2. Candidate generator:
 - **Planner**: breaks the question into smaller sub-questions that are easier to solve. Each sub-question can be translated into a partial SQL (i.e., an intermediate step);
 - **SQL synthesizer**: An LLM-based agent that actually composes SQL statements (candidates) based on the plan and retrieved info.
- 3. Query fixer: “*self-reflects*” on the previously generated query, using feedback such as syntax error details or empty result sets to guide the correction process
- 4. Selection agent: Choose the best SQL query from the set of viable candidates.



Self-Reflection in LLM Agent

- An **Actor** model generates text and actions conditioned on the state observations;
- An **Evaluator** model assesses the quality of the generated outputs produced by the Actor;
- An **Self-Reflection** model generates verbal reinforcement cues to assist the Actor in self-improvement.



Algorithm 1 Reinforcement via self-reflection

Initialize Actor, Evaluator, Self-Reflection:
 M_a, M_e, M_{sr}
 Initialize policy $\pi_\theta(a_i|s_i), \theta = \{M_a, mem\}$
 Generate initial trajectory using π_θ
 Evaluate τ_0 using M_e
 Generate initial self-reflection sr_0 using M_{sr}
 Set $mem \leftarrow [sr_0]$
 Set $t = 0$
while M_e not pass or $t < \max$ trials **do**
 Generate $\tau_t = [a_0, o_0, \dots, a_i, o_i]$ using π_θ
 Evaluate τ_t using M_e
 Generate self-reflection sr_t using M_{sr}
 Append sr_t to mem
 Increment t
end while
return

Reflexion: Language Agents with Verbal Reinforcement Learning

Noah Shinn
Northeastern University
noahshinn024@gmail.com

Federico Cassano
Northeastern University
cassano.f@northeastern.edu

Edward Berman
Northeastern University
berman.ed@northeastern.edu

Ashwin Gopinath
Massachusetts Institute of Technology
agopi@mit.edu

Karthik Narasimhan
Princeton University
karthikn@princeton.edu

Shunyu Yao
Princeton University
shunyuy@princeton.edu

Abstract

Large language models (LLMs) have been increasingly used to interact with external environments (e.g., games, compilers, APIs) as goal-driven agents. However, it remains challenging for these language agents to quickly and efficiently learn from trial-and-error as traditional reinforcement learning methods require extensive training samples and expensive model fine-tuning. We propose *Reflexion*, a novel framework to reinforce language agents not by updating weights, but instead through linguistic feedback. Concretely, Reflexion agents verbally reflect on task feedback signals, then maintain their own reflective text in an episodic memory buffer to induce better decision-making in subsequent trials. Reflexion is flexible enough to incorporate various types (scalar values or free-form language) and sources (external or internally simulated) of feedback signals, and obtains significant improvements over a baseline agent across diverse tasks (sequential decision-making, coding, language reasoning). For example, Reflexion achieves a 91% pass@1 accuracy on the HumanEval coding benchmark, surpassing the previous state-of-the-art GPT-4 that achieves 80%. We also conduct ablation and analysis studies using different feedback signals, feedback incorporation methods, and agent types, and provide insights into how they affect performance. We release all code, demos, and datasets at <https://github.com/noahshinn024/reflexion>.

1 Introduction

Recent works such as ReAct [30], SayCan [1], Toolformer [22], HuggingGPT [23], generative agents [19], and WebGPT [17] have demonstrated the feasibility of autonomous decision-making agents that are built on top of a large language model (LLM) core. These methods use LLMs to generate text and ‘actions’ that can be used in API calls and executed in an environment. Since they rely on massive models with an enormous number of parameters, such approaches have been so far limited to using in-context examples as a way of teaching the agents, since more traditional optimization schemes like reinforcement learning with gradient descent require substantial amounts of compute and time.

Preprint. Under review.

References

- https://github.com/pemagrg1/AI_class2022/blob/main/book/Artificial-Intelligence-A-Modern-Approach-4th-Edition-1-compressed.pdf
- <https://www.promptingguide.ai/research/llm-agents>
- <https://code.visualstudio.com/docs/copilot/ai-powered-suggestions>
- <https://code.visualstudio.com/docs/copilot/chat/copilot-chat>
- <https://code.visualstudio.com/docs/copilot/copilot-smart-actions>
- <https://arxiv.org/abs/2410.01943>
- <https://angelxuanchang.github.io/nlp-class/assets/lecture-slides/L20-LLM-agents.pdf>
- <https://arxiv.org/abs/2210.03629>
- <https://aclanthology.org/2024.acl-long.371/>